

Programmation Réseaux

Utilisation de sockets

Rémi Badonnel
Slides d'Isabelle Chrisment



1

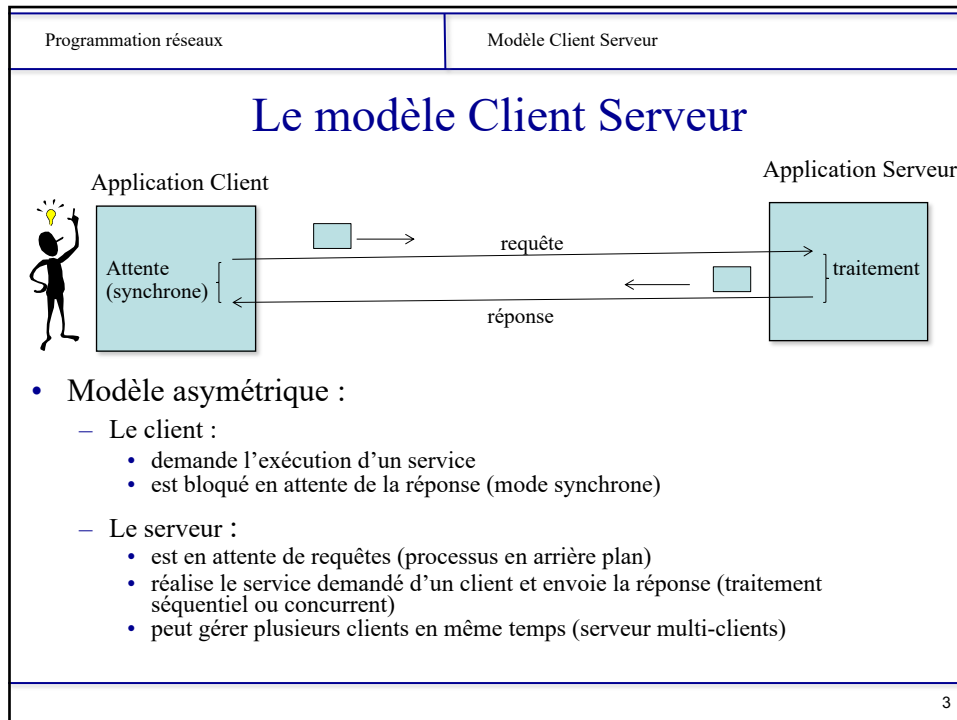
Programmation réseaux

Modèle Client Serveur

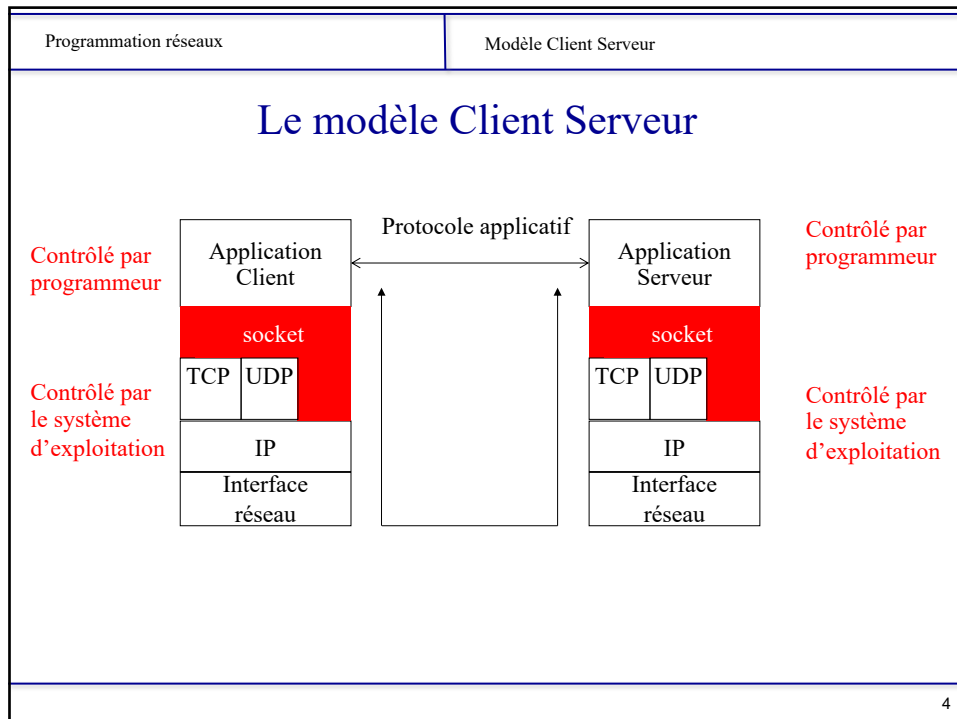
- **Programmation réseaux**
 - Modèle Client Serveur
 - Introduction socket
 - Client-Serveur TCP
 - Client-Serveur UDP
 - Entrée/sortie multiplexée : select
 - Entrée/sortie asynchrone
 - Options de socket
 - Socket raw

2

2



3



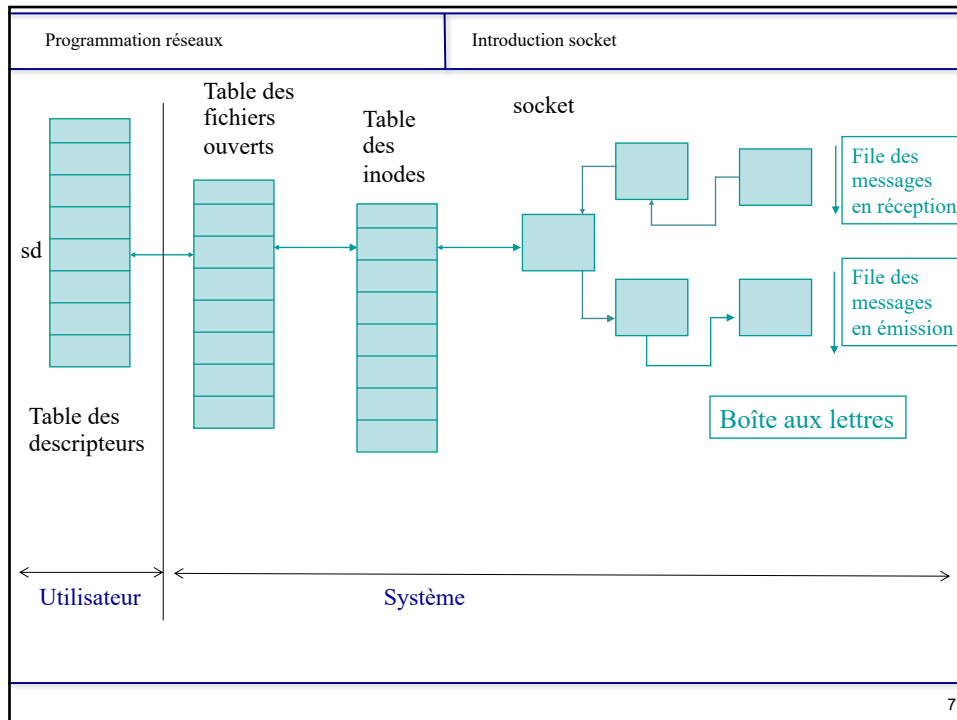
4

Programmation réseaux	
② Programmation réseaux • Modèle client serveur • Introduction socket • Client-Serveur TCP • Client-Serveur UDP • Entrée/sortie multiplexée : select • Entrée/sortie asynchrone • Options de socket • Socket raw	
5	

5

Programmation réseaux	Introduction socket
Définition d'un(e) socket • Interface de programmation (API) permettant d'envoyer et recevoir des données • Point de communication bidirectionnel – à l'intérieur d'une même machine → domaine Unix – à travers les réseaux → domaine Internet • Initialement liée à la version BSD (Berkeley) de Unix • Descripteur de socket → similaire descripteur fichier (ressources système) • Accessible par des appels systèmes	
6	

6



7

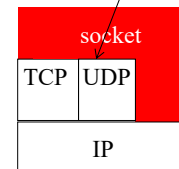
Programmation réseaux	Introduction socket
Les types de sockets	
• Précise le type de communication utilisé – SOCK_STREAM • transmission fiable, ordonnée, sans duplication • en mode connecté • autorise messages urgents • pas de préservation des limites de messages • TCP – SOCK_DGRAM – SOCK_RAW	
SOCK_STREAM socket TCP UDP IP	
8	

8

Les types de sockets

- Précise le type de communication utilisé
 - SOCK_STREAM
 - SOCK_DGRAM
 - transmission non fiable, non ordonnée, avec possibilité de duplication
 - en mode non connecté
 - préservation des limites de messages
 - UDP
 - SOCK_RAW

SOCK_DGRAM



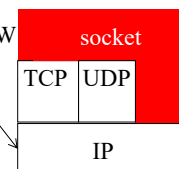
9

9

Les types de sockets

- Précise le type de communication utilisé
 - SOCK_STREAM
 - SOCK_DGRAM
 - SOCK_RAW
 - protocoles de bas niveau (super-user)
 - permet de ne pas passer par la couche transport et d'utiliser IP directement
 - ICMP

SOCK_RAW



10

10

Les familles de sockets

- Une famille ou un domaine définit :
 - La convention d'adressage ou format des adresses à utiliser
 - Le ou les protocoles réseaux supportés
- AF_UNIX (AF_LOCAL)
 - Local à la machine : domaine UNIX
- AF_INET
 - Domaine Internet et adresses IPv4
- AF_INET6
 - Domaine Internet et adresses IPv6

11

11

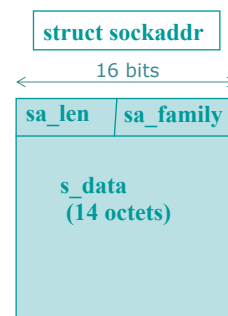
Structure générique d'un socket

Format général décrit dans
<sys/socket.h>

□

```
struct sockaddr {
    uint8_t sa_len;
    sa_family_t sa_family;
    char s_data[14];
}
```

sa_len → longueur de la structure
sa_family → AF_UNIX ou AF_INET,....
s_data → jusqu'à 14 octets pour décrire l'adresse spécifique au protocole



12

12

Structure spécifique d'un socket

- Format spécifique au protocole de communication

```
struct sockaddr_xx {  
}
```

- xx est un suffixe unique fonction du protocole
xx = **in** pour Internet (IPv4)
in6 pour Internet (IPv6)
un pour Unix

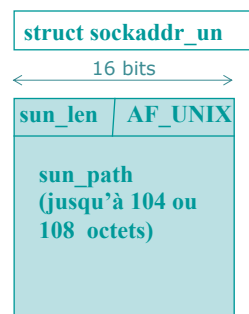
13

13

Structure dans le domaine UNIX

Format décrit dans <sys/un.h>

```
struct sockaddr_un {  
    uint8_t sun_len;  
    sa_family_t sun_family;  
    char sun_path[104];  
}  
sun_family → AF_UNIX  
sun_path → /* chemin = nom de fichier */
```



14

14

Structure dans le domaine INET

Format décrit dans `<netinet/in.h>`

```
struct sockaddr_in {
    uint8_t sin_len;
    sa_family_t sin_family;
    in_port_t sin_port;
    struct in_addr sin_addr;
    char sin_zero[8];
}
```

sin_family → AF_INET

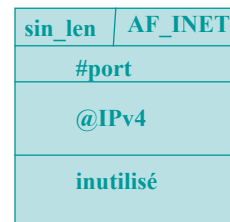
sin_port → numéro sur 16 bits dans l'ordre du réseau (network byte ordered)

sin_addr → adresse Internet sur 32 bits dans l'ordre du réseau

sin_zero → inutilisé (doit être mis à zéro)

struct sockaddr_in

← 16 bits →



15

15

Structure dans le domaine INET

Numéro de port

- Identifie de manière unique, sur une machine, un processus exécutant un service donné
 - Il existe des ports prédéfinis (cf `/etc/services`)
 - serveur web : port 80
 - serveur mail : port 25
 - appel de la fonction `getservbyname()`
 - Le port peut être choisi par le programmeur d'application
 - `>= IPPORT_RESERVED (1024)`
 - Le port peut être attribué par le système (**sin_port** ← 0)

16

16

Structure dans le domaine INET

Numéro de port – fonction getservbyname()

```
#include <netdb.h>
struct servent * getservbyname (const char *name, const char
    *proto)
name → pointeur sur la chaîne décrivant le nom du service
proto → pointeur sur la chaîne où se trouve le nom du protocole (tcp)
valeur de retour -1 si erreur

struct servent{
    char *s_name; /* nom du service*/
    char **s_aliases; /* liste alias */
    int s_port; /* numéro port (ordre réseau)*/
    int s_proto ; /* protocole utilisé */
}
```

17

17

Structure dans le domaine INET

Adresse IP

- Identifie de manière unique une machine sur un réseau
 - soit obtenue par les fonctions :
 - gethostbyname(), gethostbyname2() getipnodebyname(), getaddrinfo()
 - dans le cas d'un programme serveur la valeur INADDR_ANY est choisie : les messages peuvent être reçus sur n'importe quelle interface réseau
- Exprimée sur la forme ddd.ddd.ddd.ddd
 - exemple : 193.54.8.4
 - Dans la structure → entier format réseau

18

18

Structure dans le domaine INET

Adresse IP – fonction gethostbyname()

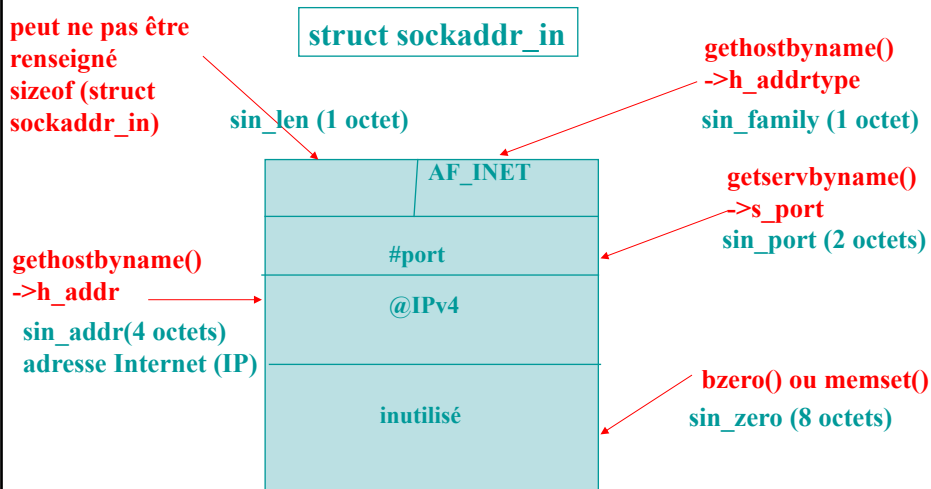
```
#include <netdb.h>

struct hostent *gethostbyname (const char *name)
name → pointeur sur la chaîne où se trouve le nom du système
valeur de retour -1 si erreur

struct hostent{
    char *h_name; /* nom machine*/
    char **h_aliases; /* liste alias */
    int h_addrtype; /* type famille */
    int h_length ; /* longueur adresse */
    char **h_addr_list; /* liste adresses */
#define h_addr h_addr_list[0]
}
```

19

19



20

20

Programmation réseaux	Introduction socket
Structure dans le domaine INET Format de représentation des données • Les machines hôtes stockent les données (entiers) dans un format – Little endian – Big endian • Pour communiquer il faut un format de transfert des données – <i>network byte order</i> : big endian	
21	

21

Programmation réseaux	Introduction socket
Structure dans le domaine INET Fonctions d'ordonnancement des octets (Byte Ordering Functions) <pre> #include <arpa/inet.h> uint16_t htons (uint16_t host16bitvalue); uint32_t htonl (uint32_t host32bitvalue); retournent une valeur dans l'ordre du réseau uint16_t ntohs (uint16_t net16bitvalue); uint32_t ntohl (uint32_t net32bitvalue); retournent une valeur dans l'ordre de la machine hôte </pre>	
22	

22

Programmation réseaux	Introduction socket
Structure dans le domaine_INET Fonctions de manipulation d'octets <pre> #include <strings.h> void bzero (void *dest, size_t nbytes); void bcopy (const void *src, void *dest, size_t nbytes); int bcmp (const void *ptr1, const void *ptr2, size_t nbytes); retourne 0 si égal #include <string.h> void *memset (void *dest, int c, size_t len); void *memcpy (void *dest, const void *src, size_t nbytes); int memcmp (const void *ptr1, const void *ptr2, size_t nbytes); retourne 0 si égal </pre> } Fonctions obsolètes	
23	

23

Programmation réseaux	Introduction socket
Structure dans le domaine_INET Fonctions de manipulation d'adresses <pre> #include <arpa/inet.h> int inet_aton (const char *strptr, struct in_addr *addrptr); retourne 1 si string valide 0 sinon in_addr_t inet_addr (const char *srcptr); retourne une valeur de 32 bits ordonnée réseau sinon INADDR_NONE char *inet_ntoa (struct in_addr inaddr); retourne un pointeur vers une chaîne contenant une adresse IP « aaa.bbb.ccc.ddd » int inet_pton(af, src, dst) int af; /* AF_INET ou AF_INET6 */ const char *src; /* l'adresse (chaîne de caract.) à traiter */ void *dst; /* le tampon où est rangé le résultat */ const char * inet_ntop(af, src, dst, size) int af; /* AF_INET ou AF_INET6 */ const void *src; /* l'adresse binaire à traiter */ char *dst; /* le tampon où est rangé le résultat */ socklen_t size; /* la taille de ce tampon */ </pre>	
24	

24

Programmation réseaux	Introduction socket
<pre> struct sockaddr_in serv_addr; struct hostent *hp; bzero(&serv_addr, sizeof(serv_addr)); memset(&serv_addr, 0, sizeof(serv_addr)); serv_addr.sin_port = htons ((ushort) atoi(argv[2])); hp = (struct hostent *)gethostbyname (argv[1]); if (hp == NULL) { fprintf(stderr, "%s: %s non trouve dans in /etc/hosts ou dans le DNS\n", argv[0], argv[1]); exit(1); } serv_addr.sin_family = hp->h_addrtype ; /* AF_INET */ serv_addr.sin_addr = * ((struct in_addr *) (hp->h_addr)); printf ("IP address: %s\n", inet_ntoa (serv_addr.sin_addr)); </pre>	
25	

25

Programmation réseaux	Client-Serveur TCP
② Programmation réseaux • Modèle client serveur • Introduction socket • Client-Serveur TCP • Client-Serveur UDP • Entrée/sortie multiplexée : select • Entrée/sortie asynchrone • Options de socket • Socket raw	
26	

26

Programmation réseaux	Client-Serveur TCP
-----------------------	--------------------

Dialogue Client-Serveur TCP

Création de socket

Ouverture de dialogue

Echange de données

Fermeture de socket

27

27

Programmation réseaux	Client-Serveur TCP
-----------------------	--------------------

Type de Dialogue TCP/IP

- transmission fiable , ordonnée, sans duplication
- en mode connecté
- autorise messages urgents
- pas de préservations de limites de messages

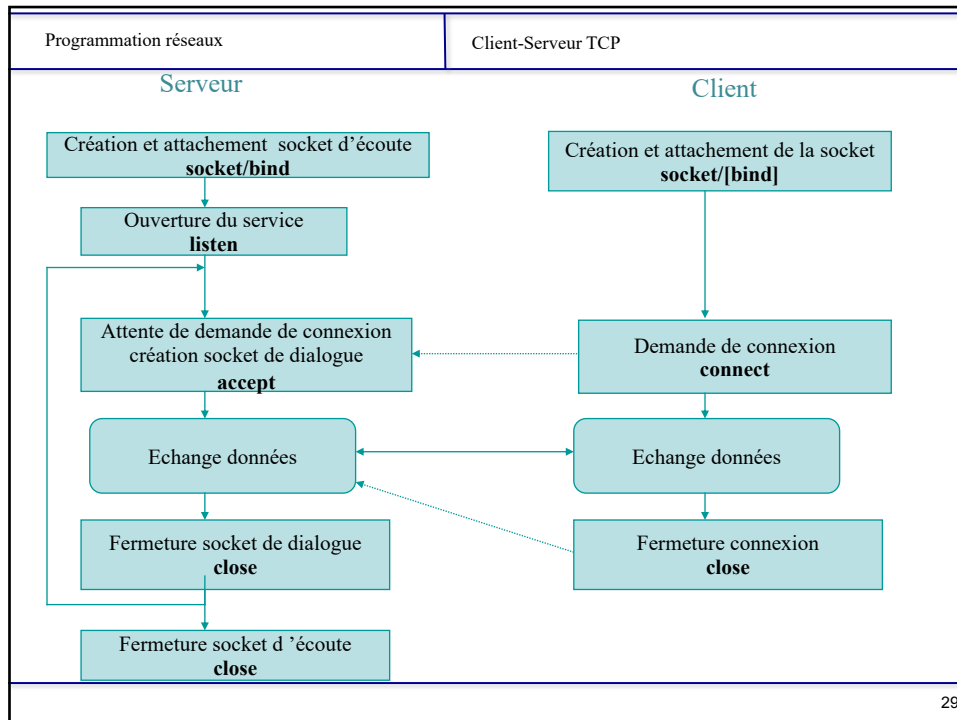


Mode de communication TCP/IP (SOCK_STREAM)

Transmission Control Protocol/Internet Protocol

28

28



29

Programmation réseaux	Client-Serveur TCP
Création d'un socket (C/S)	
<pre> #include <sys/types.h> #include <sys/socket.h> int socket (int domain, int type, int protocol) domain → [AF_UNIX ou AF_INET] → PF_UNIX ou PF_INET type → SOCK_STREAM protocol → 0 par défaut valeur de retour → -1 si erreur sinon descripteur de socket </pre>	
<pre> int serverSocket; /* * Ouvrir socket (socket STREAM) */ if ((serverSocket = socket(PF_INET, SOCK_STREAM, 0)) <0) { perror ("erreur socket"); exit (1); } </pre>	

30

Attachement/nommage d'un socket (C/S)

- Association entre un socket et une adresse

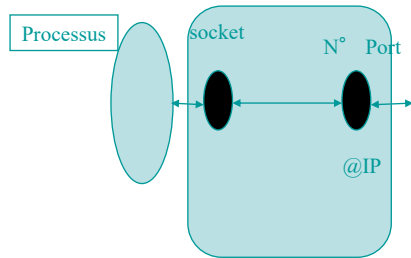
```
#include <sys/types.h>
#include <sys/socket.h>
int bind (int s, const struct sockaddr
*name, socklen_t addrlen)
```

s → descripteur retourné par socket()

name → pointeur sur une structure contenant l'adresse Internet

addrlen → longueur de l'adresse

valeur de retour → -1 si erreur



Bind : ceci est mon adresse et tous les messages reçus sur cette adresse doivent m'être délivrés

Si la machine a plusieurs adresses IP, lors du bind on pourra utiliser INADDR_ANY

31

31

```
#define SERV_PORT 2222
struct sockaddr_in serv_addr;

/*
 * Lier l'adresse locale à la socket
 */
memset (&serv_addr, 0, sizeof(serv_addr) );
serv_addr.sin_family = AF_INET ;
serv_addr.sin_addr.s_addr = htonl (INADDR_ANY);
serv_addr.sin_port = htons (SERV_PORT);

if (bind(serverSocket, (struct sockaddr *)&serv_addr, sizeof
(serv_addr) ) < 0) {
    perror ("servecho: erreur bind\n");
    exit (1);
}
```

32

32

Ouverture du service (S)

- Utilisé par le **serveur** en mode connecté pour indiquer qu'il est prêt à recevoir des demandes de connexion
- Après cet appel, seuls des paquets d'ouverture de connexion seront reçus sur ce socket

```
#include <sys/socket.h>
```

```
int listen (int s, int backlog)
```

s → descripteur retourné par socket()

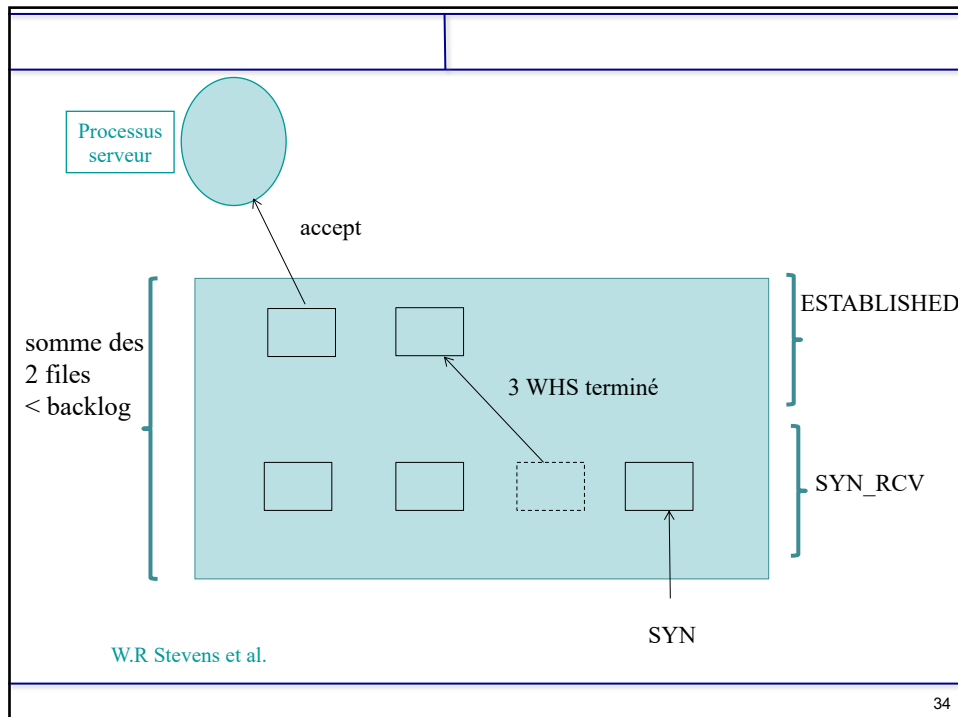
backlog → nombre de connexions simultanément en attente de traitement

valeur de retour → -1 si erreur

```
/* Paramétrer le nombre de
connexion "pending" */
if (listen(serverSocket,
SOMAXCONN) < 0) {
    perror ("servecho: erreur
listen\n");
    exit (1);
}
```

33

33



34

34

Création d'un socket de dialogue (S)

- Crée un nouveau socket qui servira au dialogue entre l'appelant et l'entité en attente de traitement de la connexion

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int accept (int s, struct sockaddr *peer,  
            socklen_t *addrlen)
```

s → descripteur retourné par socket()

peer → pointeur sur l'adresse du client qui a initié la connexion

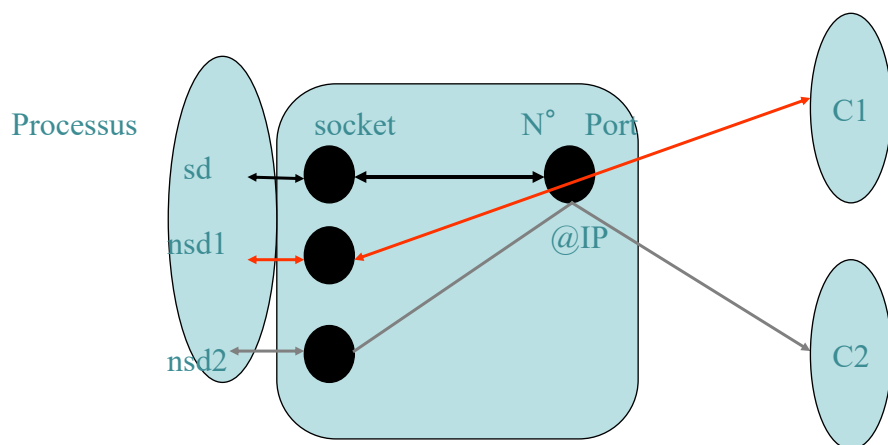
addrlen → pointeur sur la taille allouée pour peer

valeur de retour → -1 si erreur

```
int dialogSocket;  
int clilen;  
struct sockaddr_in cli_addr;  
  
clilen = sizeof(cli_addr);  
  
dialogSocket = accept(serverSocket,  
                      (struct sockaddr *)&cli_addr,  
                      (socklen_t *)&clilen);  
  
if (dialogSocket < 0) {  
    perror("servecho : erreur accep\n");  
    exit (1);  
}
```

35

35



36

36

Demande de connexion (C)

- Utilisée par un **client** (en mode connecté en général) pour établir une connexion avec un serveur distant
- Pendant cette phase : échange des paramètres du protocole TCP/IP
- Bloque l'appelant jusqu'à l'établissement de la connexion ou la génération d'une erreur
- L'établissement de connexion permet de débloquer le serveur

37

37

Demande de connexion (C)

```
#include <sys/types.h>
#include <sys/socket.h>
int connect (int s, struct sockaddr
             *servaddr, socklen_t addrlen)
```

s → descripteur retourné par socket()
servaddr → pointeur sur l'adresse du serveur vers lequel initié la connexion
addrlen → longueur de l'adresse
valeur de retour → -1 si erreur

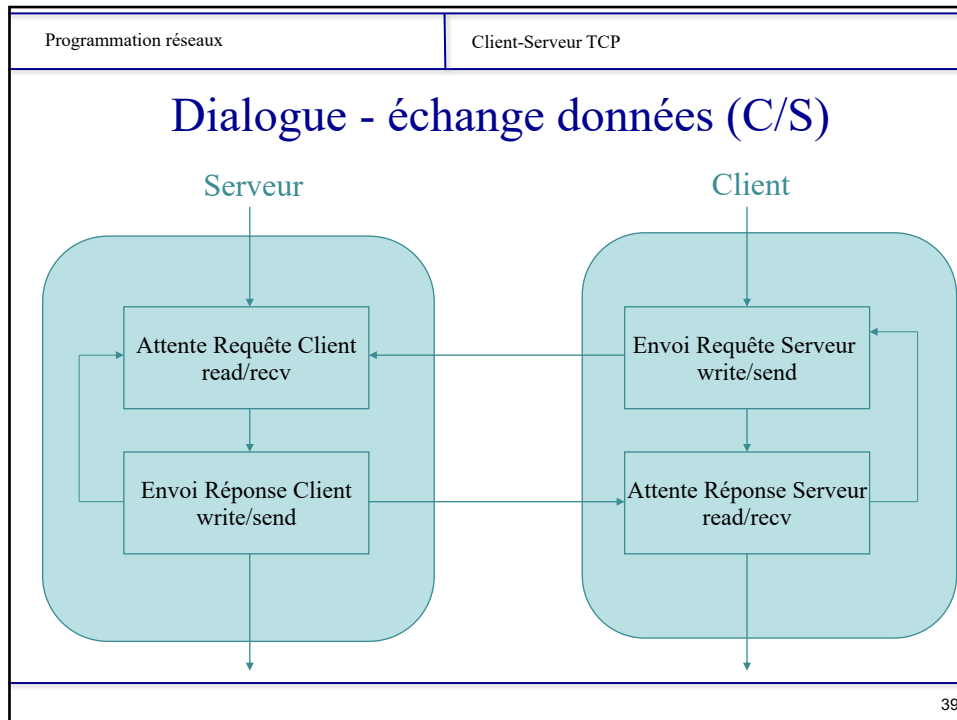
```
struct sockaddr_in serv_addr;

/*
 * Remplir la structure serv_addr avec
 * l'adresse du serveur
 */
bzero( (char *) &serv_addr,
        sizeof(serv_addr) );
serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons((ushort)
atoi(argv[2]));
serv_addr.sin_addr.s_addr = inet_addr
(argv[1]);

if (connect (sockfd, (struct sockaddr *)
&serv_addr, sizeof(serv_addr) ) < 0){
    perror ("cliecho : erreur connect");
    exit (1);
}
```

38

38



39

Programmation réseaux	Client-Serveur TCP
-----------------------	--------------------

Dialogue - échange de données (C/S)

- `send/recv` : appels système proches du `read()` et `write()` qui peuvent aussi être utilisés pour les sockets

```

#include <sys/types.h>
#include <sys/socket.h>
ssize_t send (int s, const void *buff, size_t nbytes, int flags)
ssize_t recv (int s, void *buff, size_t nbytes, int flags)
  
```

s → descripteur retourné par `accept()`
buff → pointe à l'endroit où doit être stocké le message à émettre/recevoir
nbytes → longueur du message
flags → combinaison logique des trois valeurs suivantes :

- `MSG_OOB` données urgentes
- `MSG_PEEK` lecture (sans suppression) de données entrantes
- `MSG_DONTROUTE` transmission sans routage

valeur de retour → longueur effectivement envoyée/reçue ou -1 si erreur

40

40

Fermeture d'un socket (C/S)

- Libération des ressources système

```
#include <unistd.h>
```

```
int close (int s)
```

s → descripteur de socket

valeur de retour → -1 si erreur sinon
descripteur de socket

```
#include <unistd.h>
```

```
int shutdown (int s, int how)
```

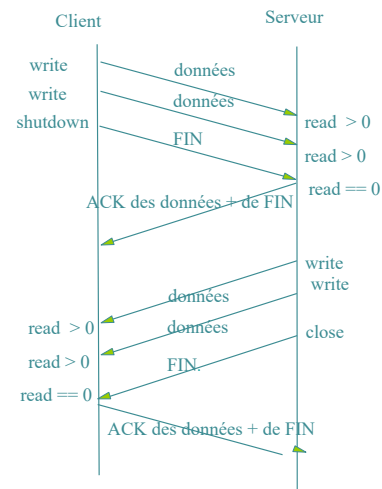
s → descripteur de socket

how → 0 fermeture en réception

1 fermeture en émission

2 fermeture en émission/réception

valeur de retour → 1 si erreur sinon
descripteur de socket



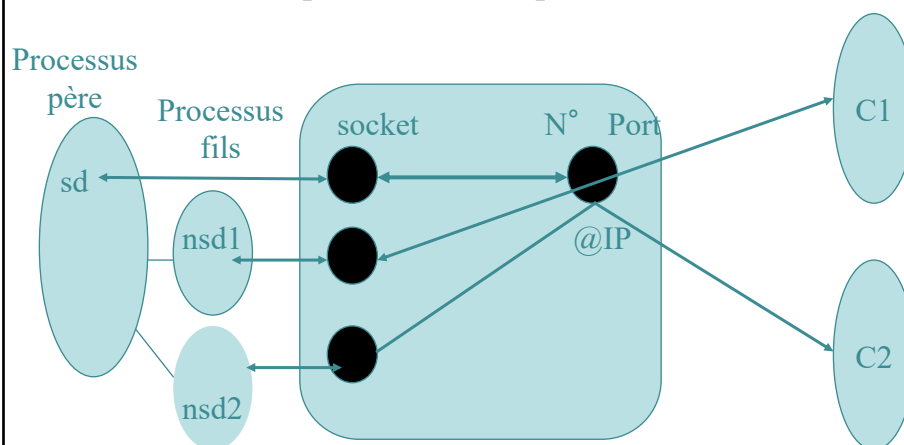
W.R Stevens et al.

41

41

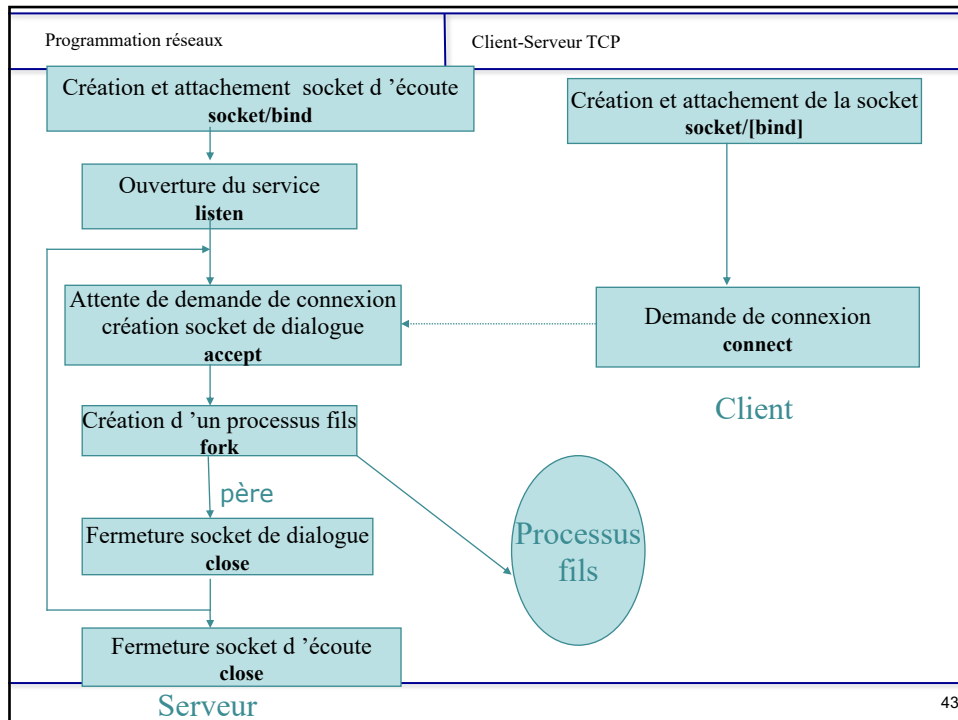
Serveur multi-clients ?

- Un serveur capable de traiter plusieurs clients à la fois

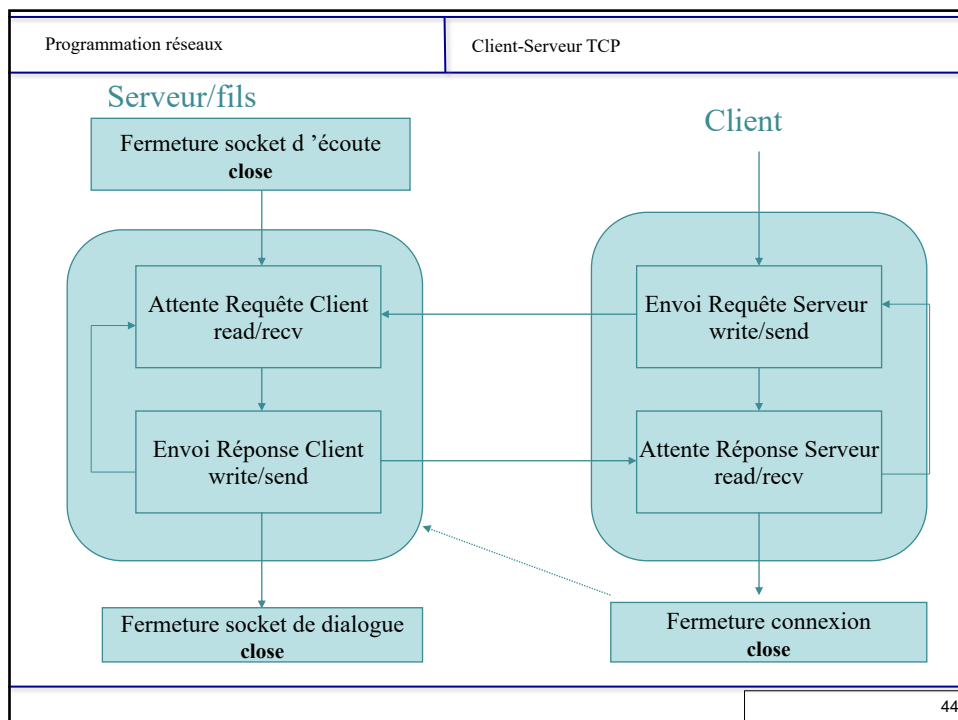


42

42



43



44

Programmation réseaux	Client-Serveur TCP
Scénario serveur concurrent	
<pre style="background-color: #f0f0f0; border: 1px solid #ccc; padding: 10px;"> int sockfd, newsockfd; if ((sockfd = socket(..)) < 0) { perror("probleme ouverture socket"); exit(1); } if bind (sockfd, ...) <0) { perror("probleme bind"); exit(1); } if listen (sockfd, 5) <0) { perror("probleme listen"); exit(1); } for(;;){ if ((newsockfd = accept (sockfd,...)) < 0) { perror("probleme accept"); exit(1); } switch (fork()) { case -1 : perror("probleme fork"); exit(1); case 0 : close (sockfd); /* fils */ traiter_requete(newsockfd); close (newsockfd); exit(0); default : close(newsockfd); /* pere*/ } /* end of switch */ } /* end of for */ </pre>	
45	

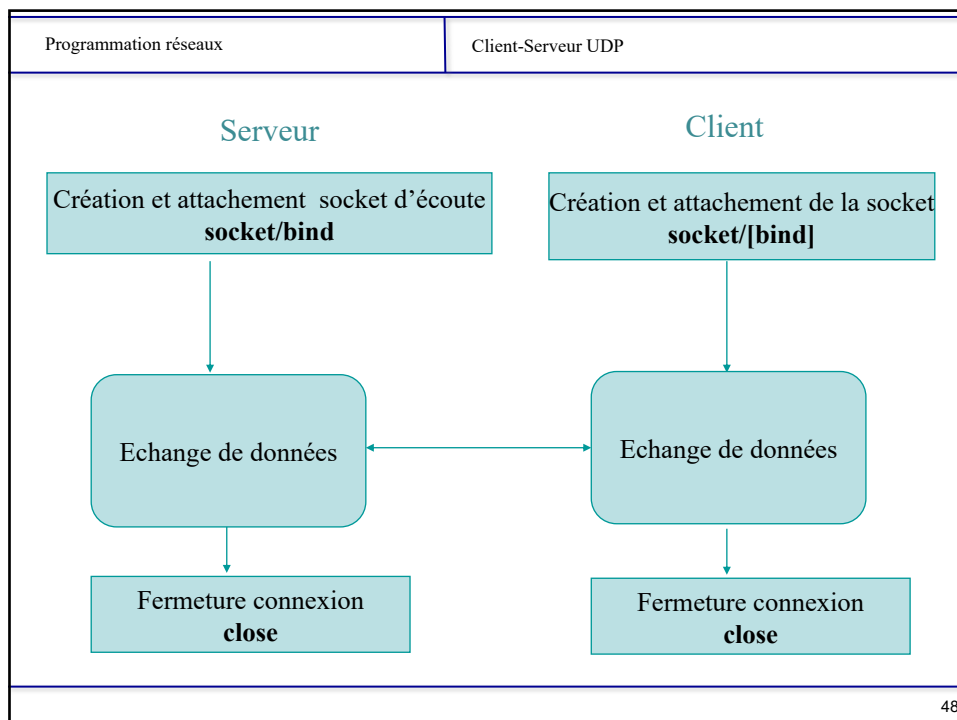
45

Programmation réseaux	
② Programmation réseaux • Modèle client serveur • Introduction socket • Client-Serveur TCP • Client-Serveur UDP • Entrée/sortie multiplexée : select • Entrée/sortie asynchrone • Options de socket • Socket raw	
46	

46

Programmation réseaux	Client-Serveur UDP
Type de Dialogue – UDP/IP – transmission non fiable – en mode non connecté – N'autorise pas les messages urgents – préservations de limites de messages Mode de communication UDP/IP (SOCK_DGRAM) User Datagram Protocol/Internet Protocol	
47	

47



48

Création d'un socket (C/S)

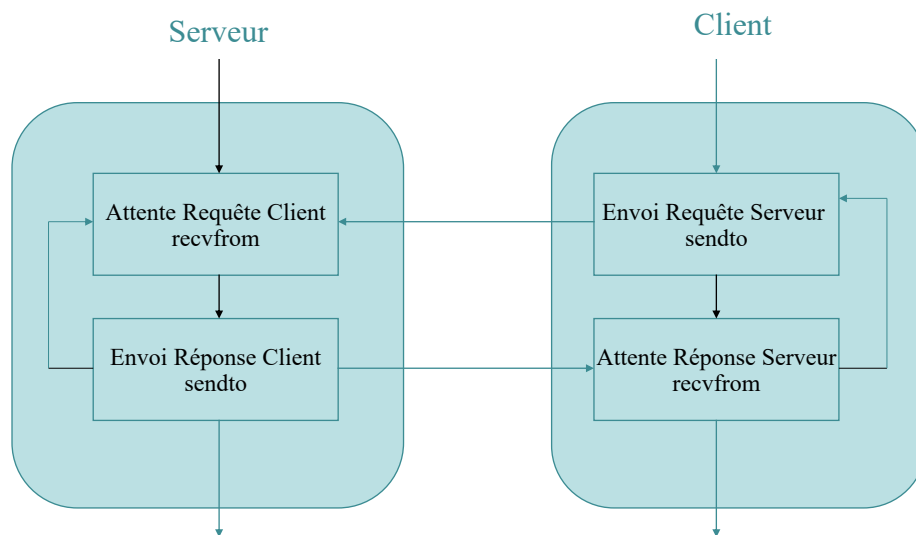
```
#include <sys/types.h>
#include <sys/socket.h>

int socket (int domain, int type, int protocol)
domain → [AF_INET]/PF_INET
type → SOCK_DGRAM
protocol → 0 par défaut
valeur de retour → -1 si erreur sinon descripteur de socket
```

49

49

Dialogue - échange données (C/S)



50

50

Dialogue - échange de données (C/S)

```
#include <sys/types.h>
#include <sys/socket.h>
ssize_t sendto (int s, const void *buff, size_t nbytes, int flags,
               const struct sockaddr *to, socklen_t tolen)
ssize_t recvfrom (int s, void *buff, size_t nbytes, int flags,
                  struct sockaddr *from, socklen_t *fromlen)
• Les 4 premiers arguments identiques au send/recv
• to/from : adresse du destinataire/émetteur du datagramme
```

51

51

② Programmation réseaux

- Modèle client serveur
- Introduction socket
- Client-Serveur TCP
- Client-Serveur UDP
- **Entrée/sortie multiplexée : select**
- Entrée/sortie asynchrone
- Options de socket
- Socket raw

52

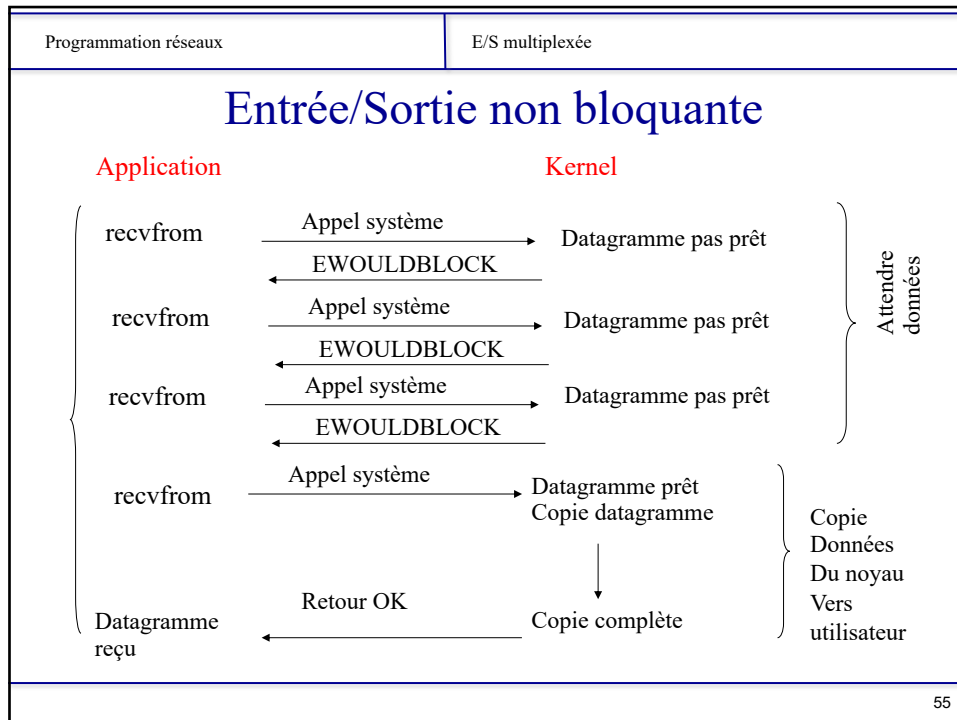
52

Programmation réseaux	E/S multiplexée
Modèles Entrée/Sortie • Quatre modèles d'entrée/sortie – E/S bloquante – E/S non bloquante – E/S multiplexée – E/S asynchrone (« signal-driven I/O »)	
53	

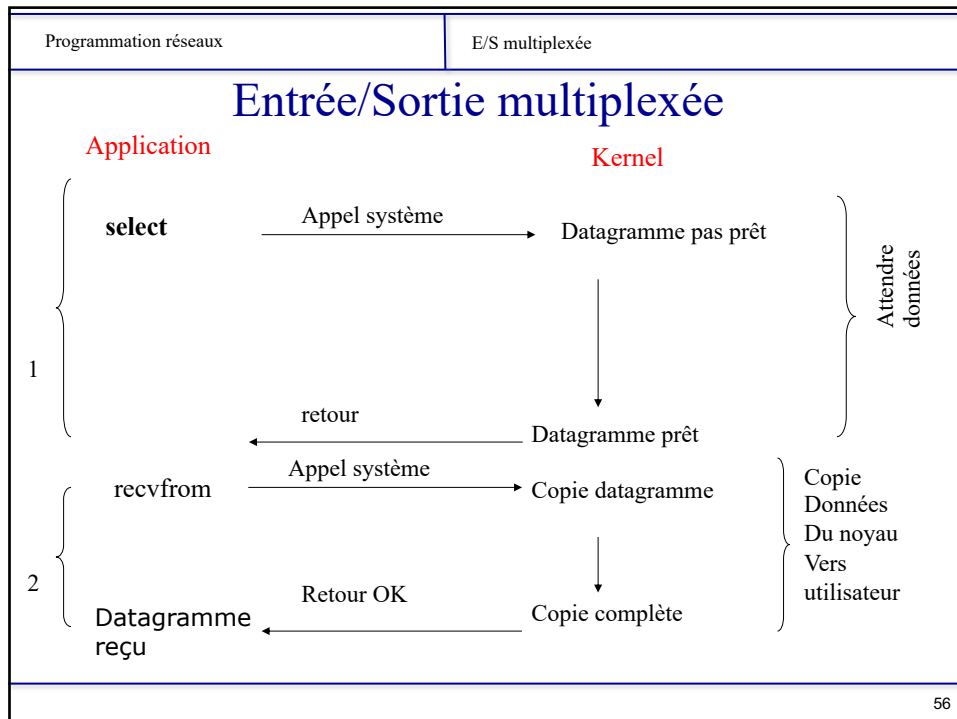
53

Programmation réseaux	E/S multiplexée
Entrée/Sortie bloquante	
Application Kernel recvfrom Datagramme reçu Appel système → ← Retour OK ↓ Datagramme prêt Copie datagramme ↓ Copie complète { Attendre données { Copie Données Du noyau Vers utilisateur	
54	

54



55



56

Programmation réseaux	E/S multiplexée
Fonction select() • Permet l'attente simultanée sur plusieurs flux d'entrée-sortie • Est utilisée : – Quand un client gère plusieurs descripteurs – Quand un client gère plusieurs sockets à la fois – Dans un serveur TCP, pour gérer à la fois le socket d'écoute et le socket de dialogue – Quand un serveur utilise à la fois UDP et TCP – Si un serveur gère plusieurs services et plusieurs protocoles • Se termine : – Quand un évènement arrive sur un flux – Au bout d'un certain temps	
57	

57

Programmation réseaux	E/S multiplexée
Fonction select() <pre>#include <sys/select.h> #include <sys/time.h> int select (int maxfdp1, fd_set *readset, fd_set *writerset, fd_set *exceptset, const struct timeval *timeout)</pre> maxfdp1 → spécifie le nombre maximum de descripteurs à tester Les descripteurs 0,1,2,... maxfdp1-1 sont testés FD_SETSIZE : nombre de descripteurs dans le type fd_set readset, writerset, exceptset → masques positionnés par l'appelant et indiquant les descripteurs pour lesquels on veut tester un prédicat en lecture (readset), en écriture (writerset), ou si une condition exceptionnelle se présente (exceptset) valeur de retour → le nombre descripteurs prêts ou 0 si timeout -1 si erreur Au retour les masques sont modifiés et indiquent les descripteurs pour lesquels le prédicat est vérifié	
58	

58

Programmation réseaux	E/S multiplexée
Fonction select() fd_set - fd_set : tableau d'entiers où chaque bit dans chaque entier correspond à un descripteur - Macros pour manipuler fd_set – void FD_ZERO (fd_set *fdset); – void FD_SET (int fd, fd_set *fdset); – void FD_CLR (int fd, fd_set *fdset); – Int FD_ISSET (int fd, fd_set *fdset);	
59	

59

Programmation réseaux	E/S multiplexée																																																						
serveur Listening Listening <pre>fd_set rset; FD_ZERO (&rset);</pre> <table border="1" style="margin: 10px auto; text-align: center;"> <tr> <td></td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td></td><td>FD_SETSIZE-1</td> </tr> <tr> <td></td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>....</td><td>0</td> </tr> </table> rset <pre>socket () + bind () + listen () FD_SET(socketecoute, &rset);</pre> <table border="1" style="margin: 10px auto; text-align: center;"> <tr> <td></td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td></td><td>FD_SETSIZE-1</td> </tr> <tr> <td></td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>....</td><td>0</td> </tr> </table> rset ← maxfdp1 = 4 → <pre>select(maxfdp1, &rset, NULL, NULL, NULL);</pre> client1 Listening Connected <table border="1" style="margin: 10px auto; text-align: center;"> <tr> <td></td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td></td><td>FD_SETSIZE-1</td> </tr> <tr> <td></td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>....</td><td>0</td> </tr> </table> rset <pre>FD_ISSET (socketecoute, &rset) → TRUE socketdialogueC1 = accept (socketecoute,..)</pre> 3WHS			0	1	2	3	4	5		FD_SETSIZE-1		0	0	0	0	0	0		0		0	1	2	3	4	5		FD_SETSIZE-1		0	0	0	1	0	0		0		0	1	2	3	4	5		FD_SETSIZE-1		0	0	0	1	0	0		0
	0	1	2	3	4	5		FD_SETSIZE-1																																															
	0	0	0	0	0	0		0																																															
	0	1	2	3	4	5		FD_SETSIZE-1																																															
	0	0	0	1	0	0		0																																															
	0	1	2	3	4	5		FD_SETSIZE-1																																															
	0	0	0	1	0	0		0																																															
60																																																							

60

Programmation réseaux	E/S multiplexée																																
<pre>FD_SET (socketdialogueC1, &rset);</pre> <table border="1" style="border-collapse: collapse; text-align: center;"> <tr> <td style="width: 20px;">0</td><td style="width: 20px;">1</td><td style="width: 20px;">2</td><td style="width: 20px;">3</td><td style="width: 20px;">4</td><td style="width: 20px;">5</td> <td style="width: 100px;"></td> <td style="width: 100px;">FD_SETSIZE-1</td> </tr> <tr> <td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td> <td>....</td> <td>0</td> </tr> </table> rset ← maxfdp1 = 5 → <pre>select(maxfdp1, &rset, NULL, NULL, NULL);</pre> client1 serveur Listening Connected <table border="1" style="border-collapse: collapse; text-align: center;"> <tr> <td style="width: 20px;">0</td><td style="width: 20px;">1</td><td style="width: 20px;">2</td><td style="width: 20px;">3</td><td style="width: 20px;">4</td><td style="width: 20px;">5</td> <td style="width: 100px;"></td> <td style="width: 100px;">FD_SETSIZE-1</td> </tr> <tr> <td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td> <td>....</td> <td>0</td> </tr> </table> rset		0	1	2	3	4	5		FD_SETSIZE-1	0	0	0	1	1	0		0	0	1	2	3	4	5		FD_SETSIZE-1	0	0	0	0	1	0		0
0	1	2	3	4	5		FD_SETSIZE-1																										
0	0	0	1	1	0		0																										
0	1	2	3	4	5		FD_SETSIZE-1																										
0	0	0	0	1	0		0																										

61

Programmation réseaux	E/S multiplexée																																
<pre>FD_SET (socketecoute, &rset);</pre> <table border="1" style="border-collapse: collapse; text-align: center;"> <tr> <td style="width: 20px;">0</td><td style="width: 20px;">1</td><td style="width: 20px;">2</td><td style="width: 20px;">3</td><td style="width: 20px;">4</td><td style="width: 20px;">5</td> <td style="width: 100px;"></td> <td style="width: 100px;">FD_SETSIZE-1</td> </tr> <tr> <td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td> <td>....</td> <td>0</td> </tr> </table> rset ← maxfdp1 = 5 → <pre>select(maxfdp1, &rset, NULL, NULL, NULL);</pre> client1 serveur Listening Connected Connected client2 <table border="1" style="border-collapse: collapse; text-align: center;"> <tr> <td style="width: 20px;">0</td><td style="width: 20px;">1</td><td style="width: 20px;">2</td><td style="width: 20px;">3</td><td style="width: 20px;">4</td><td style="width: 20px;">5</td> <td style="width: 100px;"></td> <td style="width: 100px;">FD_SETSIZE-1</td> </tr> <tr> <td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td> <td>....</td> <td>0</td> </tr> </table> rset		0	1	2	3	4	5		FD_SETSIZE-1	0	0	0	1	1	0		0	0	1	2	3	4	5		FD_SETSIZE-1	0	0	0	1	0	0		0
0	1	2	3	4	5		FD_SETSIZE-1																										
0	0	0	1	1	0		0																										
0	1	2	3	4	5		FD_SETSIZE-1																										
0	0	0	1	0	0		0																										

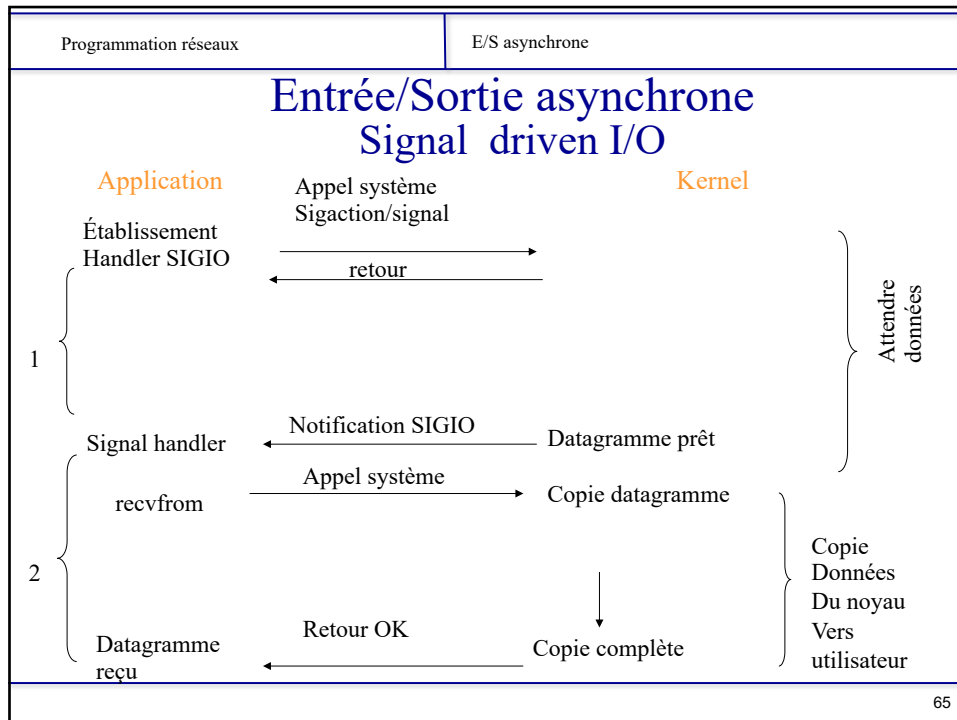
62

Programmation réseaux	E/S multiplexée
Fonction select() Contrôle de la durée d'attente <pre> struct timeval { long tv_sec ; /* secondes */ long tv_usec; /* microsecondes */ }; </pre> • Attendre toujours → pointeur NULL • Pas d'attente : retour de l'appel immédiat → champs internes de timeout initialisés à 0 • Attente limitée : attente jusqu'à arrivée d'un évènement ou fin du timer → champs internes de timeout initialisés à une valeur différente de 0 <pre> struct timeval timeout ; timeout.tv_sec=1; timeout.tv_usec=0; if (select(0, NULL,NULL,NULL, &timeout) < 0) { perror ("cliecho : erreur fgets \n"); exit(1); } </pre>	
63	

63

Programmation réseaux	E/S asynchrone
② Programmation réseaux • Modèle client serveur • Introduction socket • Client-Serveur TCP • Client-Serveur UDP • Entrée/sorties multiplexée : select • Entrée/sortie asynchrone • Options de socket • Socket raw	
64	

64



65

Programmation réseaux	E/S asynchrone
-----------------------	----------------

Socket et signaux

- 3 signaux peuvent être générés pour les sockets
 - **SIGIO**
 - Indique que la socket est prête pour une entrée ou une sortie asynchrone
 - **SIGURG**
 - Indique que des données urgentes sont disponibles en lecture
 - **SIGPIPE**
 - Indique l'écriture n'est plus possible sur le socket concerné

66

66

Socket et signaux

- Permet à un processus d'être interrompu quand un processus est disponible en lecture/écriture
 - Le noyau prévient le processus par le signal SIGIO
 - Le processus doit tout d'abord armer un « handler » pour le signal SIGIO (fonction signal/sigaction)
 - Le processus doit ensuite indiquer au noyau qu'il désire recevoir le signal SIGIO
 - fonction fcntl, commande F_SETOWN et argument pid du processus
 - Enfin le processus doit activer les entrées/sorties asynchrones sur le descripteur désiré
 - fonction fcntl, commande F_SETFL et argument O_ASYNC
- Applicable uniquement pour les terminaux et les descripteurs de sockets
- Problème ?

67

67

```
#include <signal.h>
#include <fcntl.h>
#define BUFSIZE 4096
int sigflag;
void sigio_func ()
{ sigflag = 1;}

main()
{
    int n ;
    char buff[BUFSIZE];
    signal (SIGIO, sigio_func);
    fcntl(0, F_SETOWN, getpid() );
    fcntl(0, F_SETFL, O_ASYNC) ;

    for (;;) {
        sigblock(sigmask(SIGIO));
        while (sigflag == 0) sigpause (0); /* wait for a signal */

        if ((n = read (0, buff, BUFSIZE)) > 0) {
            write (1, buff, n) ;
        }
        else if (n < 0) perror ("read error");
        else if (n == 0) exit (0); /* EOF */
        sigflag = 0;
        sigsetmask(0);
    }
}
```

68

68

Programmation réseaux	Options de socket
② Programmation réseaux • Modèle client serveur • Introduction socket • Client-Serveur TCP • Client-Serveur UDP • Entrée/sorties multiplexée : select • Entrées/sortie asynchrone • Options de socket • Socket raw	
69	

69

Programmation réseaux	Options de socket
Options sur les sockets • Plusieurs fonctions disponibles pour positionner et obtenir les valeurs des options sur les sockets – getsockopt(), setsockopt(), fcntl(), ioctl() <pre>#include <sys/types.h> #include <sys/socket.h> int getsockopt (int fd, int level, int optname, void *optval, socklen_t *optlen) int setsockopt (int fd, int level, int optname, const void *optval, socklen_t optlen)</pre> level → permet d'indiquer la couche logicielle en charge de l'option (socket, TCP, IP) : SOL_SOCKET, IPPROTO_IP, IPPROTO_TCP optname → nom de l'option optval → pointeur générique contenant la valeur de l'option optlen → contient la taille de la valeur	
70	

70

Programmation réseaux	Options de socket
(level) SOL_SOCKET • Optname SO_BROADCAST – permet d'envoyer des datagrammes en broadcast au niveau des réseaux locaux • Optname SO_RCVBUF ou SO_SNDBUF – permet de préciser la taille du buffer de réception ou émission des sockets TCP/UDP • Appel avant ou après connect(côté client) ? • Appel avant ou après listen(côté serveur) ?	
<pre> int on=1; setsockopt (sockfd, SOL_SOCKET, SO_BROADCAST,&on, sizeof(on)); int size = 4096; setsockopt (sockfd, SOL_SOCKET, SO_RCVBUF, &size,sizeof(size)); </pre>	
71	

71

Programmation réseaux	Options de socket
(level) SOL_SOCKET • Optname SO_REUSEADDR / SO_REUSEPORT – permet d'indiquer qu'une même adresse locale peut être réutilisée • Optname SO_LINGER – spécifie comment la fonction close se comporte <pre> struct linger{ int l_onoff; /* 0 → off */ int l_linger; /* temps en sec */ } </pre> Comportement par défaut (l_onoff == 0) : la fonction close retourne immédiatement	
72	

72

Programmation réseaux	Options de socket
(level) IPPROTO_IP • Optname IP_OPTIONS – permet d'utiliser les options de l'entête IP • Optname IP_TTL – permet de préciser TTL qui va être utilisé par le système – dans le cas du multicast → IP_MULTICAST_TTL • Optname IP_HDRINCL – permet d'utiliser les sockets RAW	
73	

73

Programmation réseaux	Options de socket
(level) IPPROTO_TCP • Optname TCP_MAXSEG – spécifie la taille de segment maximum • Optname TCP_NODELAY – désactive l'algorithme de Nagle	
74	

74

Programmation réseaux	Options de socket
<pre> #include <sys/types.h> #include <sys/socket.h> #include <netinet/in.h> #include <netinet/tcp.h> main() { int sockfd, maxseg, sendbuf ; socklen_t optlen ; if ((sockfd=socket(AF_INET, SOCK_STREAM,0)) <0){ perror("probleme socket"); exit(1) ; } optlen = sizeof(maxseg); if (getsockopt(sockfd, IPPROTO_TCP, TCP_MAXSEG, &maxseg, &optlen) <0){ perror("probleme getsockopt"); exit(1) ; } printf ("TCP maxseg = %d\n",maxseg); sendbuf = 16384; if (setsockopt(sockfd, SOL_SOCKET, SO_SNDBUF, &sendbuf, sizeof(sendbuf))<0){ perror("probleme setsockopt"); exit(1) ; } optlen = sizeof(sendbuf); if (getsockopt(sockfd, SOL_SOCKET, SO_SNDBUF, &sendbuf, &optlen)) <0){ perror("probleme getsockopt"); exit(1) ; } printf ("send buffer size = %d\n",sendbuf); } </pre>	
75	

75

Programmation réseaux	Socket raw
② Programmation réseaux • Modèle client serveur • Introduction socket • Client-Serveur TCP • Client-Serveur UDP • Entrée/sorties multiplexée : select • Entrée/sortie asynchrone • Options de socket • Socket raw	
76	

76

Caractéristiques d'un socket raw

- Permet à un processus de lire et d'envoyer des paquets de contrôle (ICMP, IGMP)
- Permet à un processus de lire et d'envoyer des datagrammes IP avec un champ *protocol* non traité par le noyau
- Permet à un processus de construire sa propre entête IPv4 (options socket IP_HDRINCL)

77

77

Création

```
int sockfd;

if ( (sockfd=socket(AF_INET, SOCK_RAW, protocol)) <0){
    perror("probleme socket");
    exit(1) ;
}
```

- **protocol** → constante définie dans <netinet/in.h> comme IPPROTO_ICMP
- Création uniquement en mode super utilisateur

78

78

Programmation réseaux	Socket raw
Création	
<pre style="border: 1px solid black; padding: 10px;">const int on =1: if (setsockopt(sockfd, IPPROTO_IP, IP_HDRINCL, &on, sizeof(on)) <0){ perror("probleme setsockopt"); exit(1) ; }</pre>	
• Positionnement de l'option IP_HDRINCL si en-tête IP est inclus dans la donnée • Fonction bind peut être appelée mais rare - Pas de concept de numéro de port, uniquement adresse locale • Fonction connect peut être appelée mais rare	
79	

79

Programmation réseaux	Socket raw
Emission	
• Utilisation de sendto en spécifiant l'adresse IP destination • Si l'option IP_HDRINCL n'est pas positionnée – L'adresse de début de la donnée = le 1er octet suivant l'en-tête IP • Le noyau fragmente les paquets raw qui excèdent la taille du MTU de l'interface sortante	
80	

80

Programmation réseaux	Socket raw
Réception • Utilisation de recvfrom • Les paquets TCP et UDP reçus ne sont jamais passés à une socket RAW • La plupart des paquets ICMP sont passés à une socket RAW une fois que le noyau a fini de traiter le message ICMP – Les implémentations dérivées de BSD passent tous les paquets ICMP sauf echo request, timestamp request et address mask request. • Tous les paquets IGMP sont passés à une socket RAW une fois que le noyau a fini de traiter le message IGMP • Tous les datagrammes IP avec un champ <i>protocol</i> non reconnu par le noyau sont passés à une socket RAW	
81	

81

Programmation réseaux	Socket raw
Réception • Si un datagramme arrive en fragments : – Attente de tous les fragments • Tests à vérifier pour obtenir un datagramme IP – Si <i>protocol</i> $\neq 0$ spécifié à la création du socket RAW alors • Champ <i>protocol</i> du datagramme reçu == champ <i>protocol</i> spécifié – Si une adresse IP locale est liée au socket RAW par <i>bind</i> alors • Adresse destination du datagramme reçu == adresse locale – Si une adresse distante est spécifiée par <i>connect</i> alors • Adresse source du datagramme reçu == adresse distante	
82	

82

Programmation Réseaux

Utilisation de sockets

Rémi Badonnel
Slides d'Isabelle Chrisment



UNIVERSITÉ
DE LORRAINE